Introdução a Multi-agent Systems com CrewAI

Asimov Academy

ASIMOV

Conteúdo

01. A grande oportunidade existente hoje no mercado de IA **3**
O Ecossistema de IA: Três Camadas Interligadas . . . 3
Camada dos Provedores de Modelos . . . 3
Camada de Infraestrutura Cloud . . . 3
Camada de Infraestrutura de Hardware . . . 4
A ligação entre as camadas . . . 4
A camada mais importante (o topo da pirâmide) – As Aplicações com IAs . . . 4
Uma analogia para o momento que vivemos . . . 4
Onde estarão as grandes empresas do futuro? . . . 5
Exemplos práticos de aplicações . . . 5
A trilha de Multi-Agent Systems . . . 6

02. O que é um agente? **7**
2022 – Surgimento dos Modelos de LLM . . . 7
Limitações dos modelos . . . 7
2023 – Primeiras aplicações compostas de IA . . . 7
2024 – Soluções baseadas em Agentes . . . 8
Características de um Agente . . . 9
A Arquitetura ReAct . . . 9
Mudança no fluxo de ação do sistema? . . . 10
Quando utilizar a lógica de agentes? . . . 10

03. Como pensar ao trabalhar com modelos de linguagem? **12**
Entendendo o Zero-Shot . . . 12
A Abordagem Zero-Shot e suas Limitações . . . 12
Repensando a interação com modelos de linguagem . . . 13
A Importância do raciocínio e da decomposição do problema . . . 13
Resultados equivalentes com arquiteturas adequadas . . . 14

04. O que é um Multi-agent System? **15**
Limitações de um Agente . . . 15
Solução através dos Multi-agent System . . . 15
Benefícios . . . 16
Arquitetura de sistemas Multi Agentes . . . 16
1. Agente Simples com Ferramentas . . . 16
2. Network de Agentes . . . 17
3. Arquitetura de Supervisor . . . 18

4. Supervisor com Agentes como Ferramentas 19
5. Estrutura Hierárquica 20
Resumo das arquiteturas 21

05. Principais frameworks para Aplicaçõe IA **24**
A linguagem de programação 24
A Importância dos frameworks 24
Overview dos principais frameworks 25
CrewAI 25
Swarm 25
LangGraph 25
AutoGen 26

06. O que você vai aprender nesta trilha? **27**

01. A grande oportunidade existente hoje no mercado de IA

Como aproveitar as oportunidades que as IAs estão trazendo hoje para revolucionar o seu negócio, o seu ambiente de trabalho e a forma como você realiza suas atividades?

Das diversas notícias sobre o mercado de IA surgindo, se fala sobre OpenAI, DeepSeak, novos modelos, novas arquiteturas e termos que se repetem, como *agents*, *multi agent systems*, *tools*, assistentes, etc., mas as grandes perguntas ainda permanecem sem resposta: **como isso vai impactar o meu trabalho. E mais, como me inserir neste novo mercado?**

Todos chegam aqui **empolgados** com a revolução que estamos vivenciando e, ao mesmo tempo, **confusos**, pois é muita informação. É natural querer entender onde estão as oportunidades dessa transformação – e é justamente isso que quero começar a explicar para vocês.

O Ecossistema de IA: Três Camadas Interligadas

Como vocês sabem, esta é a trilha de Multi Agent Systems. Legal, mas o que isso significa? Talvez você ainda não faça ideia. Na verdade, quem já entende o que são os multi agent systems provavelmente não está assistindo a uma aula; está criando uma aplicação de IA neste exato momento, pois é aí que reside a principal demanda da indústria.

Mas vamos dar um passo atrás para explicar como está estruturada hoje a indústria das IAs generativas, para que você entenda onde você pode se inserir!

Camada dos Provedores de Modelos

Grande parte das notícias que vemos é sobre os provedores dos modelos LLM (*Large Language Models*, como o ChatGPT). Nessa categoria se enquadram OpenAI, DeepSeak, Claude, Google, entre outros, todos tentando gerar modelos cada vez mais eficientes (com menor custo) e com maior qualidade de resposta.

Quando falamos de custo, temos dois principais:

- **O custo de treinar um modelo;**
- **O custo de rodá-lo.**

Camada de Infraestrutura Cloud

Tanto o treinamento quanto a execução de um modelo pronto necessitam de grandes centros de infraestrutura. É aí que entram os provedores de infraestrutura: AWS, Google Cloud, Azure, entre outros, que fornecem os recursos necessários para rodar esses modelos.

Camada de Infraestrutura de Hardware

Por fim, na camada mais básica desse ecossistema, estão os desenvolvedores de hardware, que fornecem os chips e as placas de vídeo essenciais para o processamento eficiente dos modelos, como a Nvidea que tem sido tanto falada.

A ligação entre as camadas

Essas três camadas estão completamente interligadas. Quanto mais o mundo utiliza as IAs generativas, maior é a demanda sobre a infraestrutura dos provedores de cloud e, consequentemente, mais chipsets e placas de vídeo serão necessários.

Por isso, vemos uma clara valorização das ações e dos resultados financeiros de empresas como a Nvidia e a Amazon (dona da AWS).

A camada mais importante (o topo da pirâmide) – As Aplicações com IAs

Esse é o panorama básico. Mas onde nós entramos?

Sobre esse organograma existe um nível ainda acima das três camadas, que chamamos de **Camada das Aplicações de IA**.

Nas três camadas inferiores, a capacidade de atuação é limitada. Treinar um modelo, rodá-lo de forma eficiente ou criar uma nova placa de vídeo são empreendimentos caros e acessíveis a poucos players. Porém, criar uma aplicação de IA está ao alcance de todos nós!

Uma analogia para o momento que vivemos

Eu costumo comparar essa situação com o advento da eletricidade. Os players da base da pirâmide forneciam uma tecnologia nova e extremamente poderosa para as massas. Quando a luz elétrica foi popularizada, ninguém imaginava que, no futuro, desenvolveríamos eletrodomésticos, televisões, celulares, computadores – todas essas aplicações só foram possíveis porque tínhamos uma tecnologia de base: a eletricidade.

Hoje, vivenciamos algo similar com as IAs. É difícil mensurar o impacto que essa popularização terá em nosso dia a dia. Não posso afirmar quais aplicações se tornarão populares a partir de agora, mas tenho certeza de que muitos novos produtos, empresas – uma indústria inteira – surgirão a partir dessa nova tecnologia.

Onde estarão as grandes empresas do futuro?

Você percebe que as grandes empresas de hoje não são aquelas que produzem energia elétrica, mas sim as que utilizam esse poder para criar aplicações com alto valor agregado. Na economia das IAs, veremos o mesmo movimento: o valor, hoje alocado na base da pirâmide, começará a se deslocar (e já está se deslocando) para o topo, onde serão desenvolvidas aplicações práticas para problemas reais.

Atualmente, estamos apenas engatinhando na utilização das IAs. Muitas empresas já incorporam pequenas aplicações de IA em sua rotina, mas ainda vejo uma mentalidade bastante limitada – estamos apenas replicando funções humanas. No entanto, o espaço para pensar diferente e criar aplicações únicas está aí.

Na pior das hipóteses, as IAs tornarão as empresas e nós mesmos muito mais produtivos. Na melhor das hipóteses, poderemos resolver problemas que antes eram considerados insolúveis.

Exemplos práticos de aplicações

Para exemplificar concretamente as aplicações que podemos criar, vou citar alguns exemplos que utilizamos aqui na Asimov:

- **Oráculo:**
  Um app que reúne todo o conhecimento necessário da nossa empresa, utilizado tanto no atendimento aos clientes quanto no treinamento da equipe de vendas. Com ele, realizamos simulações completas de atendimento e aprimoramos suas respostas com base nos dados coletados das nossas vendas.
- **Analisador de Currículos:**
  Uma aplicação simples que filtra os currículos com maior potencial para as vagas em aberto na empresa. Um processo que antes consumia um dia inteiro do tempo de um funcionário agora dura apenas 5 minutos.
- **Criador de Blog Post:**
  Uma ferramenta que acessa conteúdos das nossas aulas – como transcrições de áudios, posts antigos e apostilas – para gerar novos conteúdos autorais para o nosso blog.

Essas são soluções simples, mas práticas, que aumentam significativamente a eficiência da nossa equipe. Todas partiram de dores que nossa equipe sentia e culminaram em soluções alinhadas a essas necessidades. Pare um momento agora e pense: quais processos repetitivos você realiza diariamente? Onde você ou seus colegas desperdiçam tempo?

Esse é o foco que eu preciso que você mantenha durante esta trilha: construir aplicações é resolver problemas – alguns que já nos afligem hoje, e outros que ainda nem percebemos que temos.

A trilha de Multi-Agent Systems

E como você construirá essas soluções? É isso que veremos nesta nova trilha da Asimov: a criação de aplicações que não apenas resolvem problemas, mas que alavancam o poder das IAs por meio de duas principais arquiteturas:

1. **Agentes**
2. **Colaboração** – criando, assim, aplicações de múltiplos agentes.

Mas o que é um agente e como se dá essa colaboração? Esse é o conteúdo da próxima aula.

Sejam muito bem-vindos à trilha de Multi Agent Systems e vamos construir aplicações de IA juntos!

02. O que é um agente?

Ao final desta aula, espero esclarecer para vocês o que é um agente e qual a diferença entre uma aplicação baseada em agentes e uma aplicação simples que utiliza apenas modelos de linguagem. Para isso, vamos traçar uma linha do tempo, começando em 2022.

2022 – Surgimento dos Modelos de LLM

Neste ano, a OpenAI começou a lançar seus primeiros modelos de linguagem, que revolucionaram tanto a forma de treinamento e desenvolvimento quanto, principalmente, a sua capacidade quase humana de gerar texto. Foi um grande boom, que se tornou público e impactante já em dezembro de 2022.

Limitações dos modelos

Embora impressionantes, esses primeiros modelos tinham aplicações práticas limitadas. Eles eram, em sua maioria, vistos como ferramentas para experimentação e diversão, mas apresentavam restrições fundamentais, principalmente devido a dois fatores:

- **Dados de treinamento limitados:**
  Os modelos só sabiam o que foi fornecido durante o treinamento, o que limitava seu conhecimento sobre o mundo e sua capacidade de resolver atividades específicas.
- **Baixa adaptabilidade:**
  Como eram treinados para gerar a resposta mais provável, os modelos tendiam a oferecer respostas genéricas, o que dificultava, por exemplo, a criação de um chatbot de atendimento personalizado – não havia informações sobre o produto da sua loja ou sobre o cliente com quem você estava se comunicando.

2023 – Primeiras aplicações compostas de IA

A partir de 2023, começaram a surgir aplicações compostas que combinavam os modelos de LLM com ferramentas capazes de gerar informações contextuais. Por exemplo, em um chatbot, seria possível:

- Executar uma query no banco de dados para obter informações sobre os produtos de interesse do cliente.
- Consultar o histórico de compras do cliente.
- Passar todas essas informações junto com a pergunta do cliente para o modelo de linguagem.

Esse fluxo – **query para o banco de dados → LLM → resposta gerada** – permitia superar as limitações de:

- Informação insuficiente (ao adicionar dados do contexto);
- Baixa adaptabilidade (ao ajustar a resposta conforme o histórico do usuário).

Com isso, começamos a ver modelos gerando soluções reais, que agregavam valor para as empresas, integrando de forma mais complexa os processos e as necessidades dos ambientes de trabalho.

2024 – Soluções baseadas em Agentes

Qual o porém dessa forma tradicional de criação de aplicações?

Na abordagem composta descrita, a sequência de passos – ou, em termos de programação, a lógica de controle – é pré-definida por quem desenvolve a aplicação. Por exemplo, mesmo que a informação consultada no banco de dados não seja necessária para responder à dúvida do usuário, o sistema sempre seguirá a mesma sequência: query → processamento → resposta.

Agora, imagine se, em vez de pré-definir esses passos, permitíssemos que o próprio modelo de linguagem determinasse quais são os passos ótimos, de acordo com a necessidade do usuário. Por exemplo:

- Se o usuário simplesmente disser “olá”, o sistema pode rodar o modelo de linguagem para responder diretamente.
- Se o usuário perguntar “qual o preço do produto X?”, o sistema decide, de forma autonoma, que é necessário fazer uma query ao banco de dados sobre o produto X, incluir essa informação na entrada do modelo e, só então, gerar a resposta.

Nesse cenário, o modelo passa a ser responsável por decompor o problema em subproblemas e desenvolver um plano adequado à necessidade apresentada. E este foi o avanço que vimos em 2024 com o surgimento da arquitetura de agentes, uma consequência do maior entendimento que estamos tendo quanto a utilização mais eficiente dos modelos de linguagem.

Características de um Agente

Para que fique bem claro, um sistema baseado em agentes possui três características principais:

1. **Reasoning (Raciocínio):**
   Um agente sempre passa por uma etapa inicial de análise do problema, definindo um plano para solucioná-lo. Essa capacidade de "pensar" pode ser induzida por um prompt que orienta o modelo a dividir o problema em etapas menores e mais fáceis de resolver. Atualmente, essa habilidade de raciocínio já está presente em modelos avançados, como os da linha "o" da OpenAI ou os modelos de reasoning da DeepSeak, como o R1.
2. **Act (Ação através de tools):**
   O agente executa ações utilizando ferramentas externas, as *tools*. Essas ferramentas podem fornecer informações atualizadas (por exemplo, buscar na internet, acessar um banco de dados, consultar uma API para obter informações sobre clima ou trânsito) ou permitir a interação com o mundo (como enviar um e-mail, mandar uma mensagem no WhatsApp ou salvar dados em uma base).
3. **Memory (Memória):**
   Os agentes geralmente dispõem de dois tipos de memória:
   - **Memória de curto prazo:** Relacionada ao processo atual de raciocínio. Assim como eu preciso lembrar o que acabei de dizer para dar sequência à aula, o agente guarda informações temporárias para que suas ações façam sentido.
   - **Memória de longo prazo:** Armazena o histórico completo de interação com o usuário, permitindo a construção de um contexto adequado ao longo do tempo.

A Arquitetura ReAct

Da combinação de *Reasoning* e *Act* surge a primeira arquitetura famosa de agentes, chamada **ReAct**. Segue um exemplo de prompt ReAct, que estimula um modelo de linguagem a raciocinar antes de agir:

```
Responda às seguintes perguntas o melhor que puder. Você tem acesso às seguintes

ferramenta 1: descrição da ferramenta
ferramenta 2: descrição da ferramenta
ferramenta n: descrição da ferramenta
```

```
Utilize o seguinte formato:

Pergunta: a pergunta de entrada que você deve responder
Pensamento: você deve sempre pensar sobre o que fazer
Ação: a ação a ser tomada (deve ser uma das ferramentas previamente descritas)
Entrada da Ação: a entrada para a ação
Observação: o resultado da ação
... (esse ciclo Pensamento/Ação/Entrada da Ação/Observação pode se repetir quant
Pensamento: Agora eu sei a resposta final
Resposta Final: a resposta final para a pergunta de entrada original

Comece!

Pergunta: [Pergunta do usuário]
Pensamento: [Memória de curto prazo]
```

O resultado dessa abordagem foi surpreendente. Pesquisadores perceberam que, com essa estratégia, modelos mais simples (como o GPT-3.5) podiam obter respostas melhores do que modelos de gerações superiores (como o GPT-4), apenas ajustando seu comportamento para agir de forma agentica.

Mudança no fluxo de ação do sistema?

Com arquiteturas como a ReAct, o nosso sistema passa a ter um fluxo muito similar ao que os seres humanos utilizam ao realizar uma tarefa. Ao receber uma pergunta, primeiramente pensamos e criamos um plano de ação. Em seguida, executamos a ação, observamos o resultado e refletimos: será que, com as informações obtidas, já é possível resolver o problema? Se sim, uma resposta é retornada ao usuário; se não, planejamos a próxima ação, executamos e reiteramos esse ciclo.

Quando utilizar a lógica de agentes?

Essa nova forma de construir sistemas – baseada em agentes – não invalida as abordagens programáticas tradicionais. Existe um equilíbrio a ser considerado:

- **Lógica Programática:**
  Quanto mais pré-definido for o fluxo de ações, maior o controle sobre a saída do sistema, mas menor sua autonomia.

- **Lógica Agentica:**
  Quanto mais o sistema delegar a definição dos passos ao modelo, maior será sua autonomia, mas o controle sobre a saída se torna menor.

Em situações onde o problema é bem definido e a entrada é previsível, uma abordagem programática pode ser mais eficiente. Por outro lado, quando o problema é mais aberto – como em um chatbot que precisa lidar com diversas formas de interação – um sistema baseado em agentes pode oferecer maior flexibilidade e melhores resultados.

Na Asimov, por exemplo, utilizamos um sistema programático para o criador de blog posts, pois as entradas e saídas são bem definidas. Já em sistemas como o Oráculo, onde a interação é mais dinâmica e as respostas dependem de múltiplas bases de dados, a abordagem agentica se mostra mais adequada.

Espero que, ao final desta aula, vocês tenham compreendido o que é um agente, suas características essenciais e como essa abordagem pode transformar a criação de aplicações de IA.

Para reforçar, quero que vocês saiam daqui com dois conceitos sólidos em mente:

1. **Duas formas de construir aplicações de IA:**
   - **Programática:** Permite um controle maior, com um fluxo pré-definido.
   - **Baseada em agentes:** Concede mais autonomia ao modelo, permitindo que ele defina os passos conforme a necessidade.
2. **Composição de um agente:**
   Um agente é composto por sua capacidade de **raciocinar**, **agir através de tools** e **guardar memória**.

Na próxima aula, vamos explorar a colaboração entre esses agentes, entendendo o conceito de sistemas multiagentes (os famosos Multi Agent Systems)!

03. Como pensar ao trabalhar com modelos de linguagem?

Esta aula é muito importante, pois vou transmitir a principal ideia que você precisa ter em mente para trabalhar bem com as IAs generativas.

Nós temos uma ferramenta extremamente potente, desenvolvida e oferecida a um custo ínfimo para toda a sociedade. Mas de que adianta ter uma Ferrari se eu nem ao menos sei dirigir? É o momento de entendermos como utilizar corretamente as IAs, extraindo o máximo de seu potencial para gerar valor para a nossa sociedade.

Entendendo o Zero-Shot

Para entender como utilizar bem uma IA, gosto primeiro de explicar como não utilizá-la. Tradicionalmente, muitos utilizam os modelos fornecendo um comando ou uma pergunta de uma única vez e esperando que eles retornem uma resposta satisfatória. Essa abordagem é conhecida como *zero-shot*. Por exemplo, podemos pedir: "Crie um artigo de blog sobre IAs generativas."

Coloquem-se na posição do modelo. Imagine que você precisa escrever um texto sobre um tema específico sem ter se preparado previamente – sem tempo para pesquisar ou refletir sobre os conceitos que gostaria de abordar ou a mensagem que deseja transmitir. Pegue uma folha em branco e tente escrever algo coerente e de qualidade sobre o tema. Você acharia essa tarefa extremamente desafiadora, não é mesmo? É exatamente isso que pedimos a um modelo de linguagem ao utilizarmos a abordagem zero-shot: exigimos que ele produza, instantaneamente, um conteúdo de qualidade, sem a oportunidade de raciocinar ou se organizar previamente.

Assim como um ser humano precisaria se informar, planejar e revisar seu texto para aprimorar sua qualidade, um modelo de linguagem também enfrenta grandes dificuldades com prompts zero-shot. Embora ele possa produzir resultados razoáveis, essa não é a forma ideal de explorar todo o seu potencial.

A Abordagem Zero-Shot e suas Limitações

Quando utilizamos essa abordagem, surgem questionamentos sobre as capacidades reais do modelo. Muitas pessoas afirmam que o chat só fornece respostas banais ou óbvias. Eu sempre digo: se você não está conseguindo obter respostas satisfatórias com um modelo de linguagem, é porque você ainda não sabe usá-lo da forma adequada.

As duas principais limitações que observo ao usar a abordagem zero-shot são:

- **Respostas genéricas:**
  O modelo tende a gerar respostas padronizadas que, embora possam estar corretas em um

nível básico, não atendem a necessidades específicas. São respostas sem nuances e que não refletem necessariamente a situação concreta ou as expectativas do usuário. Isso ocorre porque os modelos são treinados para gerar a resposta mais provável, o que, naturalmente, a torna genérica.

- **Respostas simplistas:**
  A resposta pode se tornar rasa, frequentemente contendo erros básicos de lógica e coerência. Isso é um resultado natural de tentar executar uma tarefa muito complexa de uma única vez. Assim como, se escrevemos um texto de uma só vez, é inevitável que vários erros passem despercebidos, um processo iterativo de planejamento, criação e revisão se faz necessário para aprimorar a qualidade.

Repensando a interação com modelos de linguagem

Quero que vocês imaginem um modelo de linguagem como se fosse uma pessoa extremamente boa em cumprir ordens, porém muito literal. Ao pensar dessa forma, podemos adaptar nossa interação com o modelo de maneira mais eficiente.

Torna-se, então, natural criar uma interação que introduza um processo iterativo de raciocínio. Damos ao modelo tempo para pensar: quanto mais tempo ele tiver para dividir a tarefa em pequenas etapas e executar suas ações, melhor será a qualidade da resposta. Em contraste com o zero-shot, temos a técnica de *reasoning* – ou raciocínio iterativo.

Raciocínio Iterativo:
Esta abordagem envolve dividir o problema em partes menores, permitindo que o modelo utilize seu potencial de raciocínio para resolver cada subproblema de forma mais eficaz. É semelhante a como um ser humano aborda problemas complexos: pensando, quebrando-os em etapas e utilizando as informações e análises necessárias em cada fase.

A Importância do raciocínio e da decomposição do problema

A estrutura de um bom prompt – de uma boa interação com um modelo – deve começar incentivando o modelo a raciocinar antes de agir. Práticas como o *sequenciamento estrutural* são fundamentais. Em vez de solicitar uma resposta diretamente, peça ao modelo que primeiro considere o problema. Por exemplo:

- **Sequenciamento Estrutural:**
  “Antes de responder à pergunta sobre o produto Y, você pode me dar sua visão sobre os aspectos a serem considerados?”

Outra abordagem poderosa é a divisão em etapas:

- **Divisão em Etapas:**
  "Identifique os passos necessários para calcular o preço de venda do produto, considerando o custo de produção e análises de mercado. Em seguida, forneça a resposta."

Resultados equivalentes com arquiteturas adequadas

Recentemente, surgiram modelos que já incorporam a capacidade de raciocínio, como o DeepSeek R1 e o OpenAI O3. Esses modelos integram técnicas de raciocínio que já utilizamos há mais de um ano, e, naturalmente, os resultados são melhores, pois permitem que o modelo pense antes de agir.

Entretanto, é importante ressaltar que podemos alcançar resultados equivalentes – e até melhores – em certos casos ao utilizar boas arquiteturas e estratégias adaptadas. Ao empregar arquiteturas adequadas com modelos mais poderosos, o potencial de escalabilidade e eficácia aumenta consideravelmente.

O ponto principal é que, ao utilizar um modelo de linguagem de forma inteligente, você não fica limitado a esperar por um novo modelo ou por uma nova tecnologia dos criadores. Você já pode obter resultados significativamente melhores do que aqueles que utilizam essas ferramentas de forma simples e com a abordagem zero-shot.

Na próxima aula, continuaremos explorando a colaboração entre agentes e como essa interação pode ser amplificada quando aplicamos os princípios de raciocínio e decomposição do problema.

04. O que é um Multi-agent System?

Nosso objetivo nesta trilha é criar sistemas baseados em agentes, que ofereçam alta flexibilidade e autonomia para os modelos de IA.

Relembrando nossa definição de agente e suas duas principais características:

- **Raciocínio:** a capacidade de pensar, analisar e criar um plano de ação;
- **Ação:** a capacidade de agir por meio de ferramentas.

Mas qual o problema que encontramos ao criar sistemas com agentes?

Limitações de um Agente

1. **Decisão sobre ferramentas:**
   Um agente pode ter muitas ferramentas disponíveis e acabar tomando decisões inadequadas sobre qual ação executar em sua próxima interação.
2. **Crescimento do contexto:**
   O contexto de uma interação é armazenado na memória de curto prazo. Quanto maior esse contexto, pior a utilização deste pelo modelo, causando dificuldades de processamento e coerência.
3. **Necessidade de especialização:**
   Os modelos funcionam melhor quando a tarefa a ser executada é bem definida e específica. Por isso, vemos prompts que induzem os modelos a "pensar" como pesquisadores, professores, especialistas em matemática, programação, etc.

Solução através dos Multi-agent System

Para lidar com essas limitações dos sistemas agênticos, surgiu uma solução natural: dividir o sistema em múltiplos agentes menores, com escopos reduzidos, menos ferramentas e mais otimizados para tarefas específicas. Assim, surgem os **Multi-agent Systems**.

Benefícios

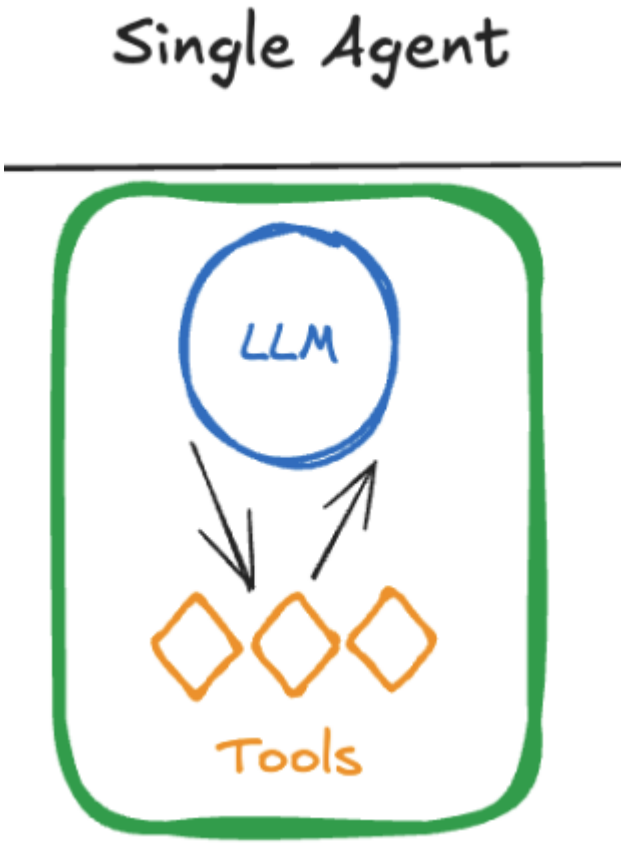
- **Modularidade:**
  Pequenos agentes, especializados em funções específicas, são mais fáceis de desenvolver, testar e podem ser reutilizados em diversos sistemas.
- **Especialização:**
  Agentes focados em determinados domínios aumentam a eficiência e a qualidade das respostas, já que cada agente é otimizado para lidar com um conjunto restrito de tarefas.

Arquitetura de sistemas Multi Agentes

Uma boa aplicação com múltiplos agentes depende de uma definição clara de como esses agentes irão colaborar. É como uma grande empresa: não adianta ter profissionais altamente capacitados se não houver um alinhamento entre suas tarefas e sua expertise, ou se eles não forem integrados adequadamente ao fluxo da organização. Existem diversas arquiteturas conhecidas, que você verá ao longo da nossa trilha.

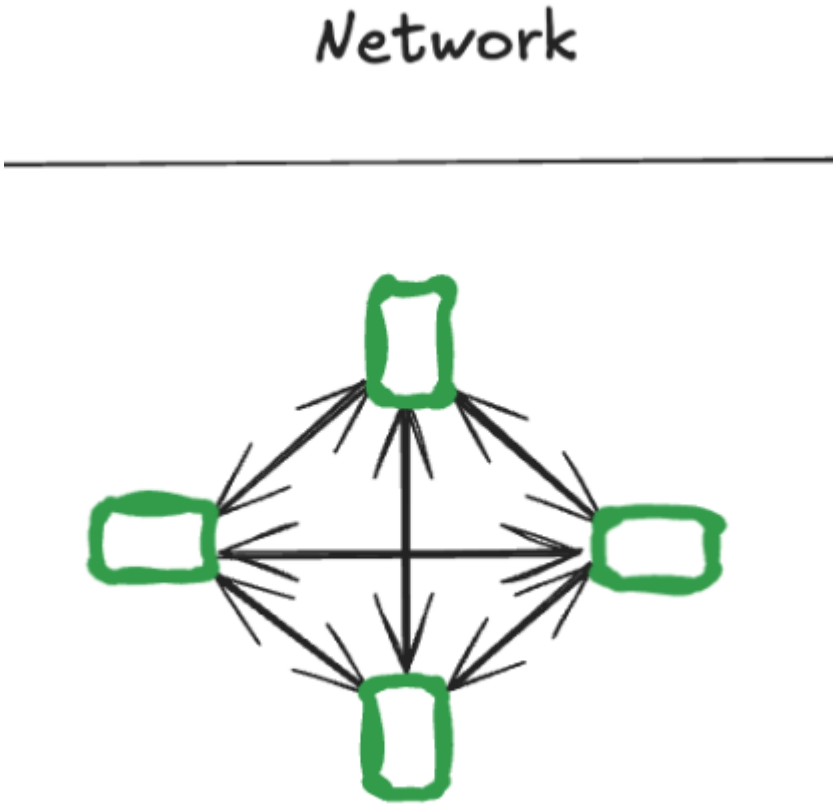
1. Agente Simples com Ferramentas

Nesta arquitetura, temos um agente único com acesso a diversas ferramentas, conforme vimos na aula anterior.



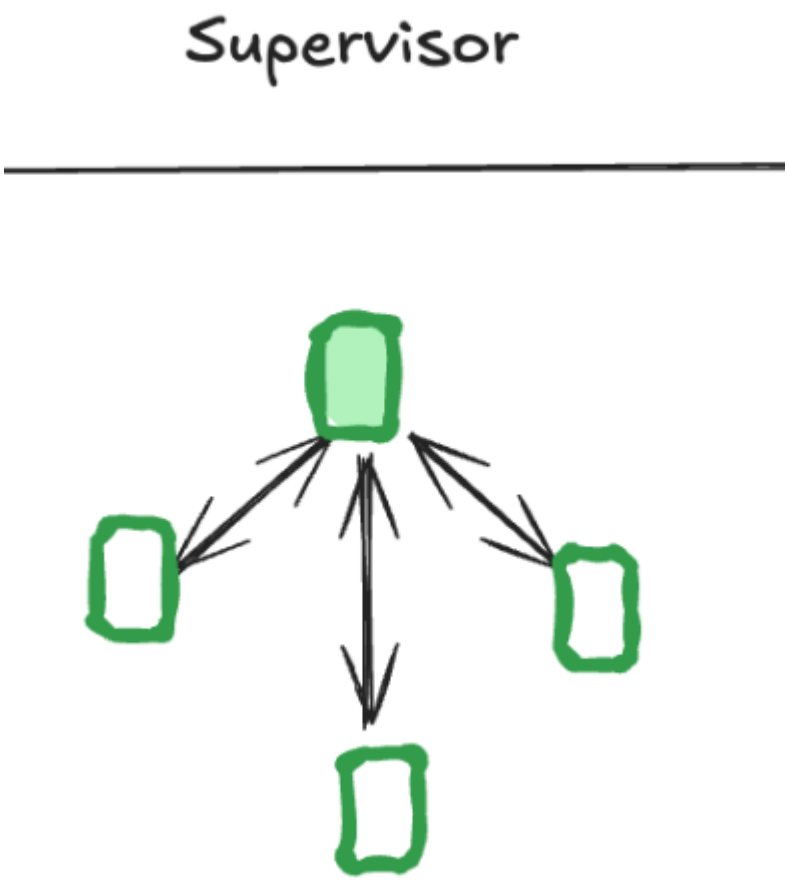
2. Network de Agentes

Aqui, múltiplos agentes, cada um com acesso a suas próprias ferramentas, decidem em conjunto qual agente será chamado em seguida. A biblioteca CrewAI, por exemplo, ganhou destaque por utilizar esse tipo de arquitetura.
Na prática, esse modelo tende a ser instável, pois a autonomia excessiva e a complexidade podem dificultar a criação de aplicações estáveis com os modelos atuais. No entanto, com a evolução dos modelos de linguagem, arquiteturas mais complexas poderão ser utilizadas de forma mais eficiente.



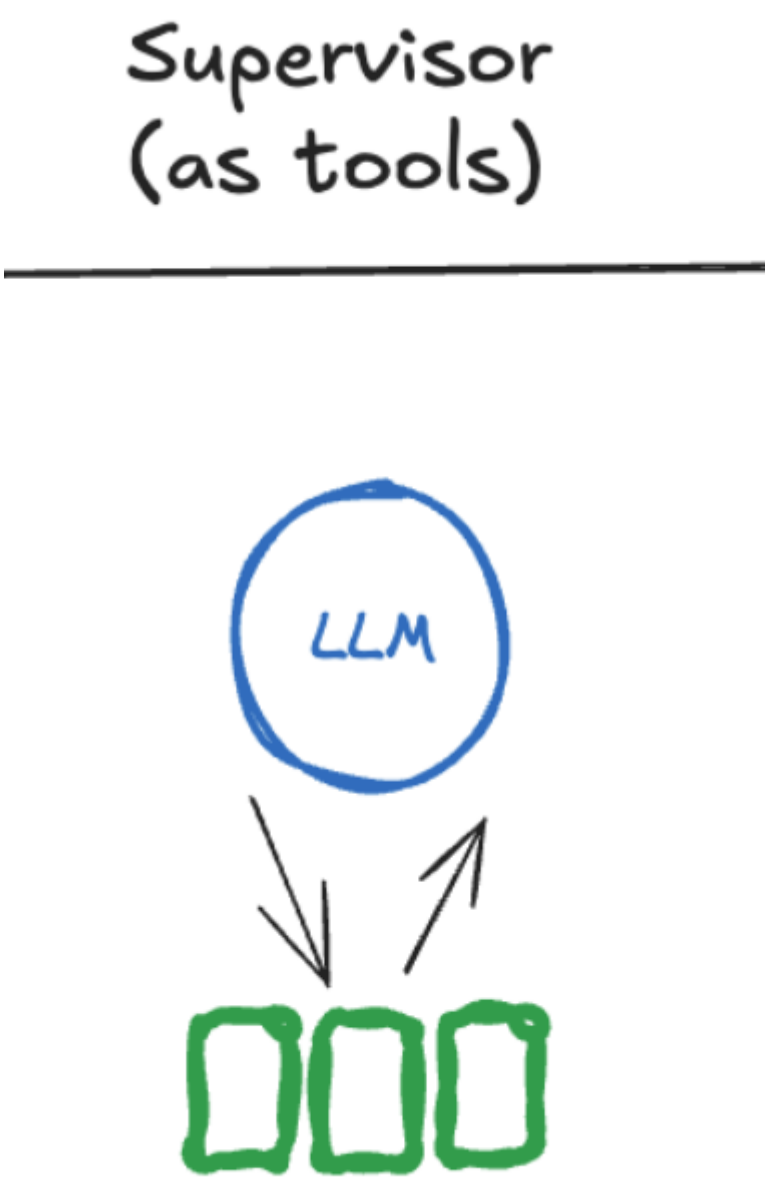
3. Arquitetura de Supervisor

Nesta arquitetura, um agente supervisor tem a função exclusiva de definir qual agente será chamado em sequência. Todos os demais agentes se concentram em executar as tarefas de seu escopo específico. Essa abordagem, embora um pouco mais rígida, já apresenta resultados muito bons e mais estáveis.



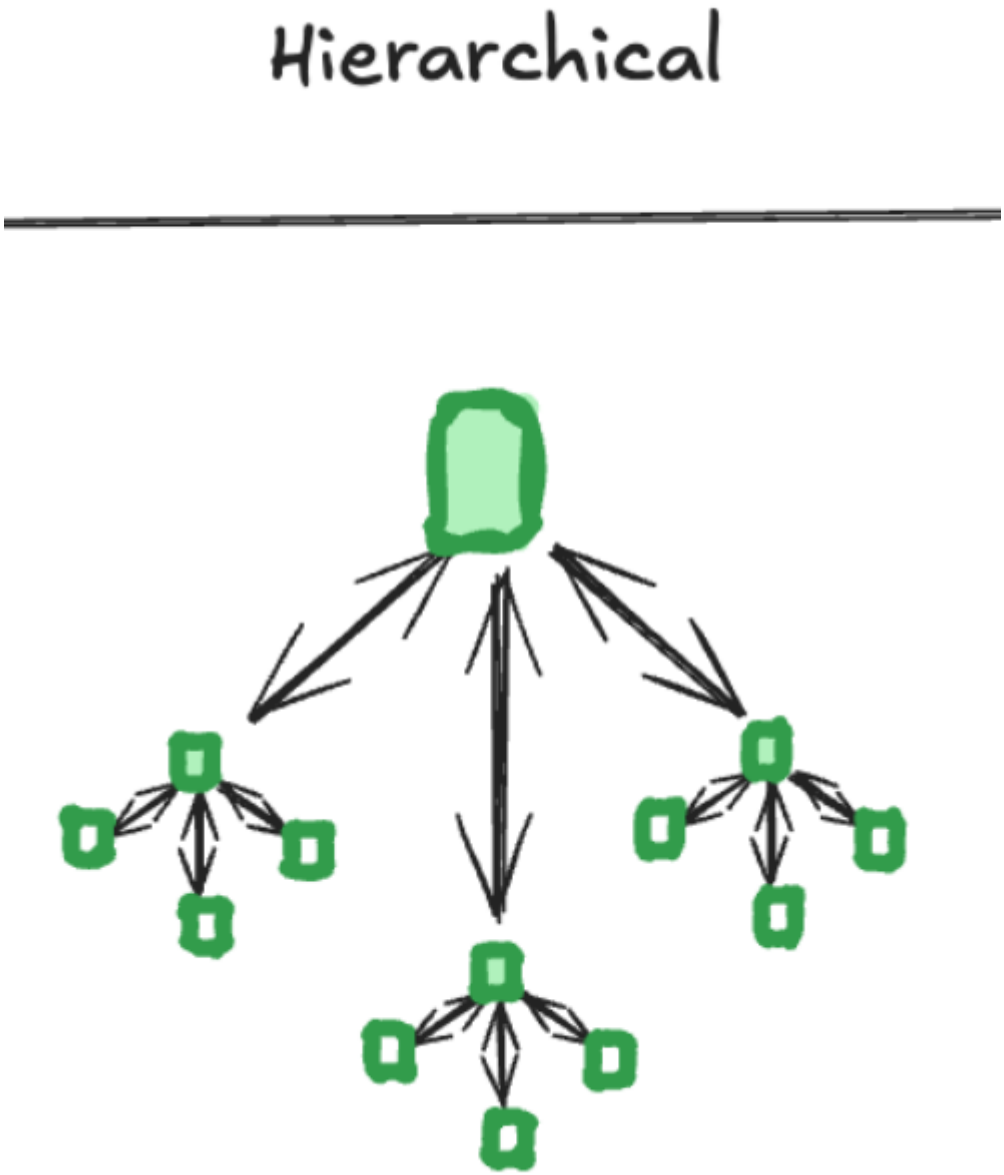
4. Supervisor com Agentes como Ferramentas

Uma versão simplificada da arquitetura de supervisor é quando os agentes especializados são incorporados como ferramentas do supervisor.
Essa é provavelmente a arquitetura mais simples e estável de um sistema multiagente, mas apresenta limitações: como os agentes funcionam como ferramentas, toda informação é transmitida por meio de argumentos, sem uma variável de estado compartilhada entre eles. Isso pode tornar o sistema mais lento e consumir mais recursos do que o necessário.



5. Estrutura Hierárquica

Nesta abordagem, criamos uma sequência de estruturas de supervisores com agentes, permitindo um crescimento potencialmente infinito do sistema. Contudo, esse aumento na hierarquia pode acarretar uma maior latência no processamento das informações.



Resumo das arquiteturas

Arquitetura	Descrição	Vantagens	Desvantagens
Agente Simples com Ferramentas	Um único agente com acesso a diversas ferramentas.	Simplicidade; facilidade de desenvolvimento e teste.	Escopo limitado; menor flexibilidade para tarefas complexas.
Network de Agentes	Múltiplos agentes autônomos que decidem em conjunto qual agente será acionado em seguida.	Alta autonomia; potencial para colaboração dinâmica entre agentes.	Instabilidade; maior complexidade; dificuldade de controle e sincronização entre os agentes.
Arquitetura de Supervisor	Um agente supervisor direciona quais agentes especializados serão acionados em sequência.	Maior estabilidade; cada agente se concentra em seu domínio, aumentando a especialização.	Menor autonomia dos agentes subordinados; dependência do supervisor para coordenação das tarefas.
Supervisor com Agentes como Ferramentas	Agentes especializados são incorporados como ferramentas dentro do supervisor.	Arquitetura simples e estável; centraliza o controle com o supervisor.	Comunicação limitada entre agentes (toda informação passa por argumentos); possível aumento no consumo de recursos.
Estrutura Hierárquica	Organização em níveis com supervisores e agentes distribuídos em uma hierarquia.	Escalabilidade; modularidade e organização clara das funções.	Aumento da latência no processamento; complexidade de gerenciamento à medida que a hierarquia se expande.

Essa tabela fornece uma visão geral dos diferentes modelos de arquiteturas multi agentes, destacando suas principais características, benefícios e desafios.

Agora você já deve estar entendendo o que é um agente e o que é um sistema multiagentes, além de conhecer suas principais arquiteturas. Lembre-se de que construímos sistemas multiagentes para criar soluções mais complexas, mas escaláveis, graças à sua modularidade, e com maior autonomia.

Um sistema multiagentes é uma fronteira do conhecimento que oferece a liberdade necessária para construir qualquer tipo de aplicação de IA que você desejar!

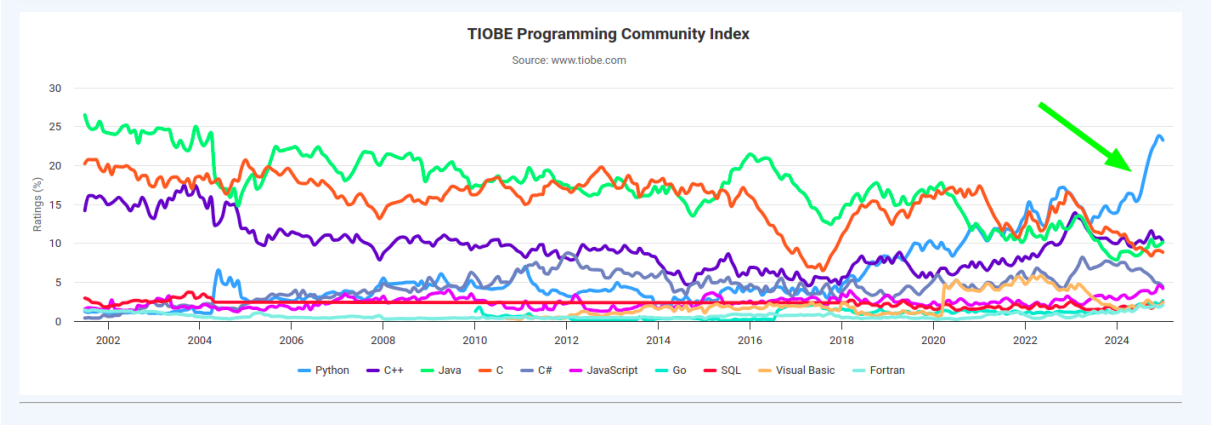
Na próxima aula, vamos abordar os principais frameworks para a construção de aplicações de IA. Espero vocês lá!

05. Principais frameworks para Aplicaçõe IA

Existem diversas formas de construir uma aplicação de IA, não apenas arquiteturas diferentes – como vimos na aula anterior – mas também ferramentas específicas que podemos utilizar na criação dessa estrutura.

A linguagem de programação

Há um elemento em que não temos opções: a linguagem de programação que utilizaremos será, com certeza, Python. Todo o desenvolvimento dessas aplicações é feito com Python, pois essa linguagem já era consolidada para Ciência de Dados.
Com o surgimento das IAs generativas, Python teve um boom de adoção entre a comunidade de programadores, como evidenciado pelo índice TIOBE – um indicador de popularidade das linguagens de programação.



Aquele crescimento, especialmente a partir de 2023, é fortemente influenciado pelo aumento do interesse e do mercado das IAs.

A Importância dos frameworks

Mesmo limitando nosso escopo à linguagem Python, existem diversas formas de criar aplicações de IA. Nos últimos anos, vimos várias bibliotecas desenvolverem frameworks para facilitar a construção de aplicações robustas.
Se você não sabe o que é um framework, pense na seguinte analogia: se você quer construir uma casa, pode fazê-la tijolo a tijolo, ou pode optar por uma casa pré-moldada, onde as paredes já vêm prontas e você simplesmente as encaixa no lugar correto.
Da mesma forma, é possível construir uma aplicação apenas com código Python puro, mas isso seria

muito mais trabalhoso – e não recomendaria essa abordagem para iniciantes – embora ofereça maior customização. Utilizar um framework permite aproveitar abstrações já desenvolvidas pela comunidade, facilitando e acelerando a criação da sua aplicação.

Overview dos principais frameworks

Nesta aula, vou apresentar um overview dos principais frameworks para o desenvolvimento de sistemas multiagentes, focando em quatro opções que se destacam atualmente:

CrewAI

O CrewAI foi um dos primeiros frameworks que vi ganhando relevância ao trabalhar com arquiteturas de agentes em rede e hierárquicas. No início, quando o conheci em 2023, achei que ele não entregava bons resultados: os agentes frequentemente se perdiam em fluxos intermináveis, gerando respostas de baixa qualidade. Porém, ele evoluiu muito nos últimos meses, tornando-se mais simples e permitindo mais controle através dos chamados *flows*, o que melhorou bastante a experiência de uso. O que torna o CrewAI interessante é sua simplicidade e objetividade, sendo uma ótima escolha para tarefas mais simples. Para quem não tem muita experiência com programação, pode ser um bom ponto de partida.

Swarm

Outro framework relevante é o Swarm, desenvolvido pela OpenAI. Ele é extremamente simples e fácil de usar, mas seu foco não é a produção de sistemas robustos e sim a parte educacional. Como ele é mais básico, não possui algumas funcionalidades que encontramos em outros frameworks, como ferramentas pré-construídas. Além disso, ele funciona exclusivamente com os modelos da OpenAI. Mesmo assim, pode ser uma boa opção para quem quer aprender os fundamentos sem lidar com muitas complexidades.

LangGraph

Já o LangGraph faz parte da família do LangChain, um dos primeiros frameworks consolidados para a criação de aplicações de IA. Enquanto o LangChain é muito bom para desenvolver um único agente, o LangGraph foi criado para permitir a orquestração de múltiplos agentes dentro de sistemas mais complexos. Seu grande diferencial é a flexibilidade e alto nível de customização, porém essa vantagem vem com um custo: a complexidade. Como o Rodrigo costuma comentar, os frameworks da família LangChain parecem ter sido desenvolvidos por programadores que vieram de outras linguagens, já

que Python é conhecido pela simplicidade e legibilidade – características que o LangGraph não segue muito bem. Ele é um framework extremamente poderoso, mas também difícil de utilizar.

AutoGen

Por fim, temos o AutoGen, um framework desenvolvido pela Microsoft. Ainda não tive muita experiência com ele, mas sei que ele se encontra em um nível intermediário de complexidade, sendo mais simples que o LangGraph, porém mais complexo que o CrewAI. Essa posição intermediária faz com que ele não seja tão customizável quanto o LangGraph, mas ainda assim seja uma opção bem competente, de acordo com relatos de colegas que o utilizaram.

Este é o ecossistema existente para a criação de aplicações de IA. Cada framework possui suas particularidades, e a escolha depende do nível de complexidade e customização que você deseja para o seu projeto.
Na próxima aula, falaremos um pouco mais sobre o framework que utilizaremos nesta trilha.

06. O que você vai aprender nesta trilha?

Um aspecto fundamental que consideramos ao criar uma nova trilha de conhecimento aqui na Asimov é oferecer um caminho claro para você!

Existem diversas formas de construir um sistema multiagente, mas não é necessário conhecer todas elas; o importante é se aprofundar em uma abordagem específica.

Entre os frameworks com os quais já trabalhamos, escolhemos o CrewAI para esta trilha, principalmente por sua simplicidade e pela flexibilidade que tem demonstrado em seu desenvolvimento nos últimos meses.

Temos conteúdos sobre outros frameworks aqui na Asimov, mas, por enquanto, não quero que você se preocupe com eles. Não adianta ser superficial em tudo; é necessário um compromisso e até uma certa fidelidade a um caminho, caso contrário ficamos sempre indecisos – e a indecisão gera estagnação.

Portanto, nesta trilha, vamos apresentar um caminho completo para construir aplicações com o CrewAI – desde a definição de um agente, de uma tarefa e de uma crew. Abordaremos as diversas formas de interação entre esses agentes ao trabalhar com o CrewAI. Também discutiremos observabilidade, utilizando o AgentOps, para que você entenda o que sua aplicação está fazendo e como aprimorá-la. E falaremos sobre deploy, de forma que, ao final, você saiba como colocar em produção a sua aplicação.

E, claro, teremos projetos que demonstrarão, de forma prática, todos os conceitos que vamos aprender. Vocês sabem que aqui o aprendizado é mão na massa – o conhecimento só se consolida quando plicamos, então desafiem-se!

Mas, antes de entrar no CrewAI, você passará pelos nossos cursos básicos de Python, pois Python é a ferramenta fundamental para qualquer desenvolvedor de IA – e será a sua também!

Então, sejam oficialmente muito bem-vindos, e vamos para a trilha!